

UNIVERSIDAD AUTÓNOMA DE BENI "JOSÉ BALLIVIAN"

Facultad de Ingeniería y Tecnología



Sistema de Registro de Tareas Colaborativas en Tiempo Real

Integrantes: 1. JOEL LEIVA CALLE
2. JOSÉ DAVID CACEREZ RIVERO
3. JOSUÉ DAVID COLQUE CANDIA

Carrera: Ingeniería de Sistemas

Docente: Ing. Hermes Rodriguez Rivero

Materia: Tecnologías Emergentes

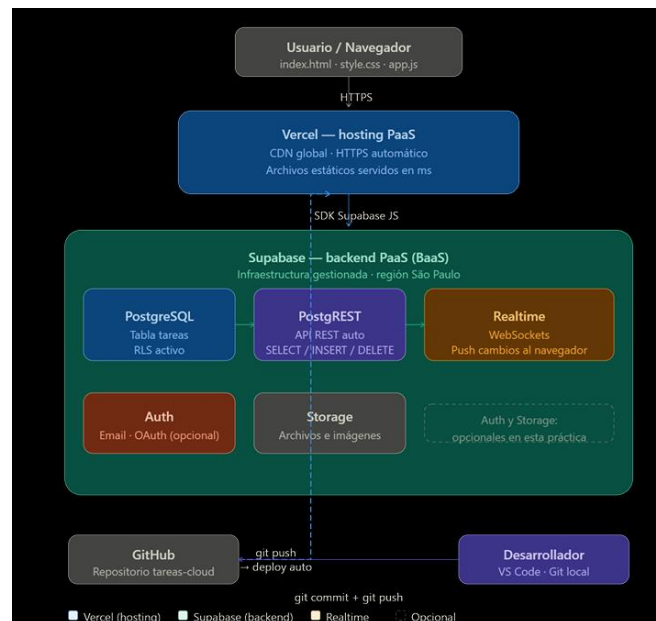
Paralelo: Séptimo

Gestión: 2026

Contenido

I.	Implementación de una Aplicación Web Real-Time bajo Arquitectura PaaS y BaaS.....	1
II.	Clasificación del modelo de servicio (IaaS/PaaS/SaaS) de cada herramienta.	2
III.	Ventajas y limitaciones encontradas.	3
IV.	FASE 1: Configuración de Supabase (PaaS – Backend en la Nube)	4
V.	FASE 2: Estructura del Proyecto en Visual Studio Code.....	5
VI.	FASE 3: Desarrollo de la Aplicación.....	6
VII.	FASE 4: Control de Versiones con Git y GitHub	8
VIII.	FASE 5: Despliegue en Vercel (PaaS de Hosting)	10
IX.	REFLEXIÓN	15

I. Implementación de una Aplicación Web Real-Time bajo Arquitectura PaaS y BaaS



La arquitectura cloud implementada para el "Sistema de Registro de Tareas Colaborativas en Tiempo Real" se basa en un modelo de Backend as a Service (BaaS) y Platform as a Service (PaaS), utilizando herramientas 100% integradas en la nube para eliminar la necesidad de gestionar servidores tradicionales.

Componentes

1. Cliente (Frontend)

El usuario interactúa con la aplicación a través de un navegador web, el cual consume archivos estáticos (HTML, CSS y JavaScript). La comunicación se realiza mediante HTTPS y utiliza el SDK de Supabase JS para conectarse directamente con los servicios del backend.

2. Capa de Hosting y Despliegue (Vercel)

Vercel actúa como la plataforma de hosting (PaaS) encargada de servir el frontend. Sus funciones principales incluyen:

- **CDN Global:** Distribuye los archivos de la aplicación para que carguen rápidamente desde cualquier ubicación.
- **HTTPS Automático:** Provee seguridad en la capa de transporte por defecto.
- **CI/CD (Integración y Despliegue Continuo):** Está vinculado con GitHub, de modo que cualquier cambio en el repositorio (a través de un git push) dispara automáticamente un nuevo despliegue en producción.

3. Capa de Backend (Supabase)

Supabase proporciona la infraestructura gestionada (BaaS/PaaS) en una región específica (como São Paulo). Sus componentes internos son:

- **PostgreSQL:** Una base de datos relacional donde se almacena la tabla de tareas. Incluye Row Level Security (RLS) para controlar el acceso a los datos.
- **PostgREST:** Una herramienta que genera automáticamente una API REST para realizar operaciones CRUD (SELECT, INSERT, UPDATE, DELETE) desde el cliente sin escribir código de backend.
- **Realtime (WebSockets):** Este componente es fundamental para la aplicación, ya que utiliza WebSockets para "empujar" (push) los cambios detectados en la base de datos directamente al navegador del usuario sin necesidad de recargar la página.
- **Auth y Storage:** Aunque son opcionales en esta práctica, forman parte de la arquitectura de Supabase para gestionar autenticación y almacenamiento de archivos.

4. Flujo de Desarrollo

El desarrollador trabaja de forma local con Visual Studio Code y utiliza Git para el control de versiones. El código se sincroniza con un repositorio remoto en GitHub, que sirve como puente para el despliegue automático hacia Vercel, cerrando el ciclo de vida del desarrollo moderno en la nube.

II. Clasificación del modelo de servicio (IaaS/PaaS/SaaS) de cada herramienta.

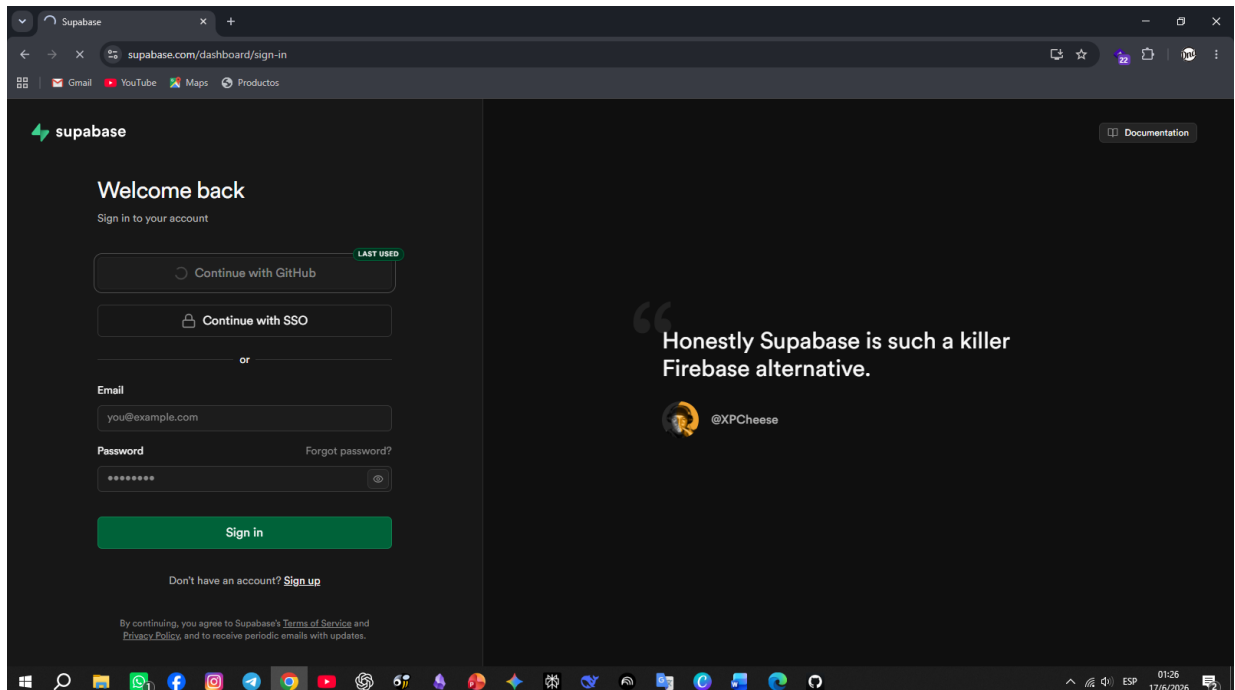
Herramienta	Clasificación (Modelo de servicio)	Submodelo	Justificación
Supabase	PaaS	BaaS (Backend as a Service)	Plataforma gestionada que ofrece base de datos PostgreSQL, autenticación y funciones en tiempo real sin gestionar infraestructura (SO, servidores o hardware).
Vercel	PaaS	—	Plataforma de despliegue que gestiona hosting, escalado automático, CDN y HTTPS, permitiendo centrarse en el desarrollo del frontend.
GitHub	SaaS	—	Servicio web listo para usar para control de versiones y colaboración, accesible sin instalar infraestructura propia.
Visual Studio Code, Git (local), Node.js	No cloud	—	Herramientas que se ejecutan localmente en la máquina del desarrollador, sin depender de servicios en la nube.
IaaS	—	—	No se utilizó infraestructura como servicio, ya que el proyecto se basó en servicios completamente gestionados (PaaS/SaaS/BaaS).

III. Ventajas y limitaciones encontradas.

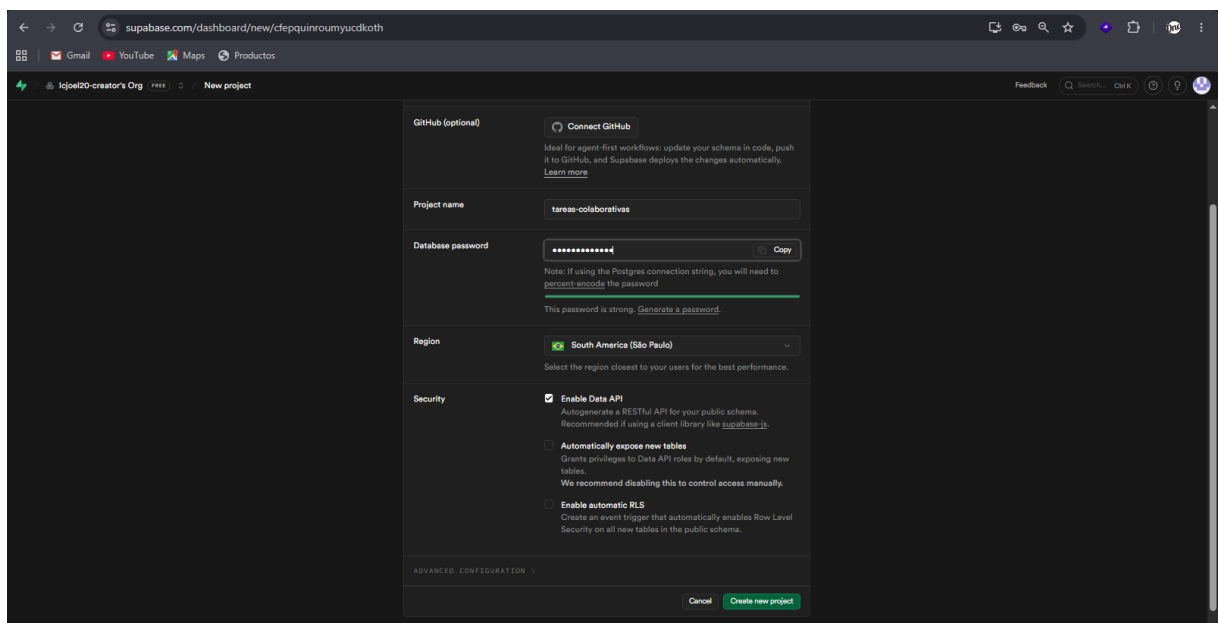
Ventajas	Desventajas
Velocidad de despliegue y desarrollo: Se elimina la gestión de servidores tradicionales, permitiendo desplegar la aplicación en minutos mediante CI/CD automático entre GitHub y Vercel.	Políticas de seguridad en producción: Uso de políticas de acceso público en la base de datos para fines académicos, lo cual no es seguro en un entorno real (requiere RLS más estrictas).
Reducción de costos: Uso de herramientas con planes gratuitos (Supabase, GitHub, Vercel), evitando inversión económica inicial.	Restricciones de capas gratuitas: Los planes Free Tier o Hobby tienen límites en almacenamiento, ancho de banda y conexiones simultáneas.
Infraestructura gestionada: No es necesario configurar sistema operativo, base de datos o hardware, ya que servicios como Supabase gestionan el backend.	Menor control del entorno: En PaaS no se tiene control total del servidor o sistema operativo como en IaaS.
Funcionalidades avanzadas "out-of-the-box": Incluye WebSockets en tiempo real y HTTPS automático sin desarrollo adicional complejo.	Dependencia del proveedor (vendor lock-in): Alta dependencia de SDKs y servicios del proveedor, lo que puede dificultar una futura migración.
Escalabilidad y disponibilidad: Uso de CDN global en Vercel que mejora la velocidad de entrega de contenido desde cualquier región.	Latencia y dependencia de internet: El rendimiento de la aplicación depende completamente de la conexión a internet y la disponibilidad de los servicios en la nube.

IV. FASE 1: Configuración de Supabase (PaaS – Backend en la Nube)

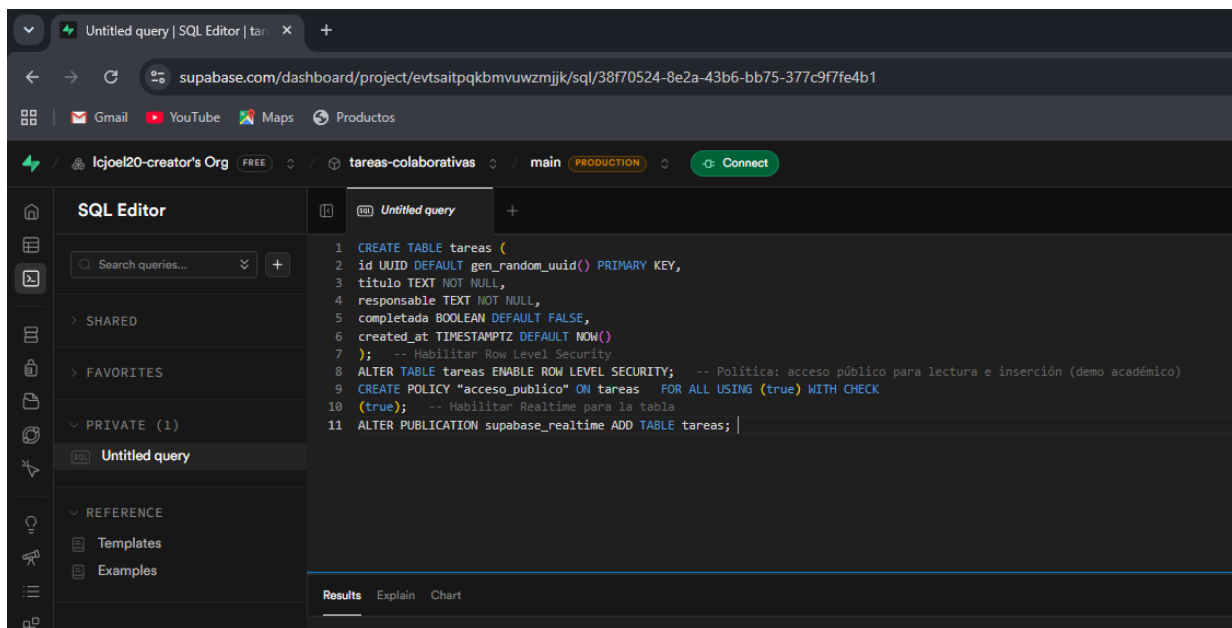
Ingresamos a <https://supabase.com> y creamos una cuenta gratuita (podemos usar nuestra cuenta de GitHub para registrarnos).



Creamos un nuevo proyecto. Elegimos un nombre para el proyecto, definimos una contraseña segura para la base de datos y seleccionamos la región más cercana (por ejemplo, South America - São Paulo).



Una vez creado el proyecto, ingresamos a la sección SQL Editor y ejecutamos el siguiente script para crear la tabla:



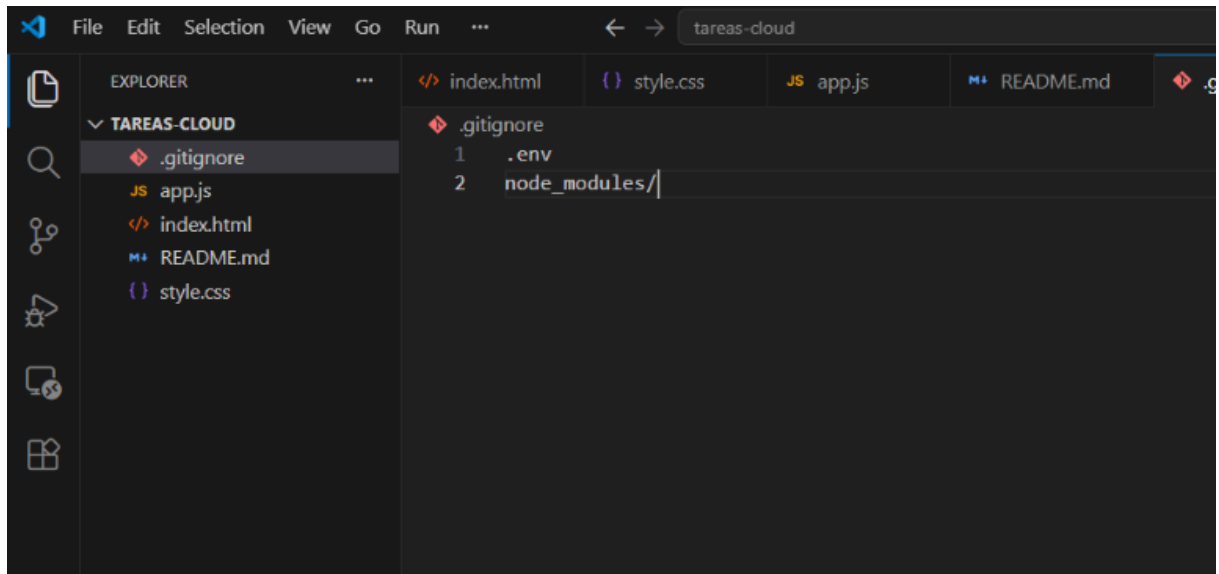
Ingresamos a la sección Project Settings > API y copiamos los valores de Project URL y anon public key. Estas serán las credenciales que utilizaremos en la aplicación.

V. FASE 2: Estructura del Proyecto en Visual Studio Code

1. Creamos una carpeta local llamada tareas-cloud.
2. Abrimos la carpeta en VS Code (File > Open Folder).
3. Creamos la siguiente estructura de archivos:

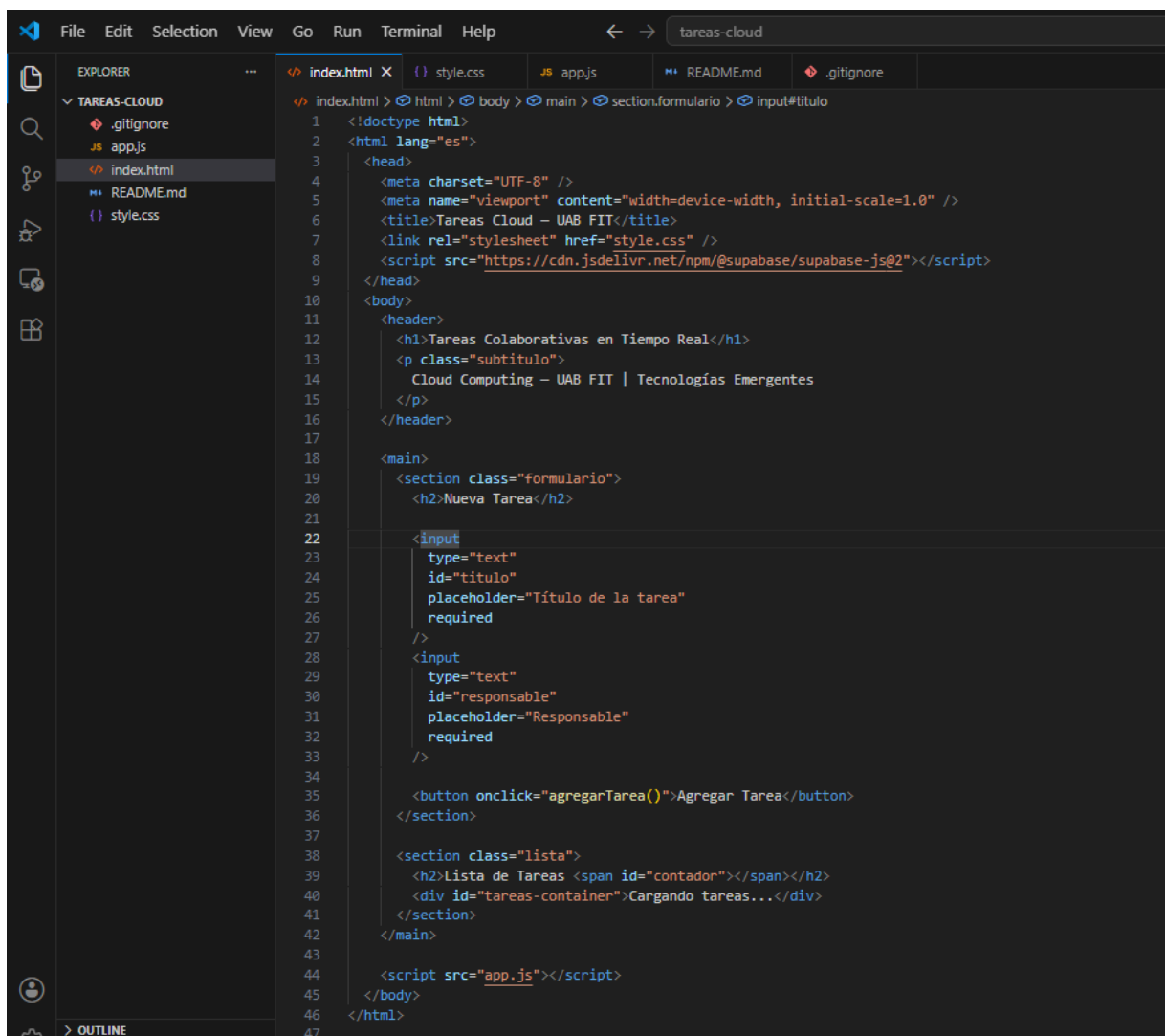


Tenemos:



VI. FASE 3: Desarrollo de la Aplicación

Creamos el archivo index.html con el siguiente contenido base:



Creamos el archivo `app.js` con la lógica de conexión a Supabase:

[illegible]

Creamos el archivo `style.css` con estilos modernos y responsivos:

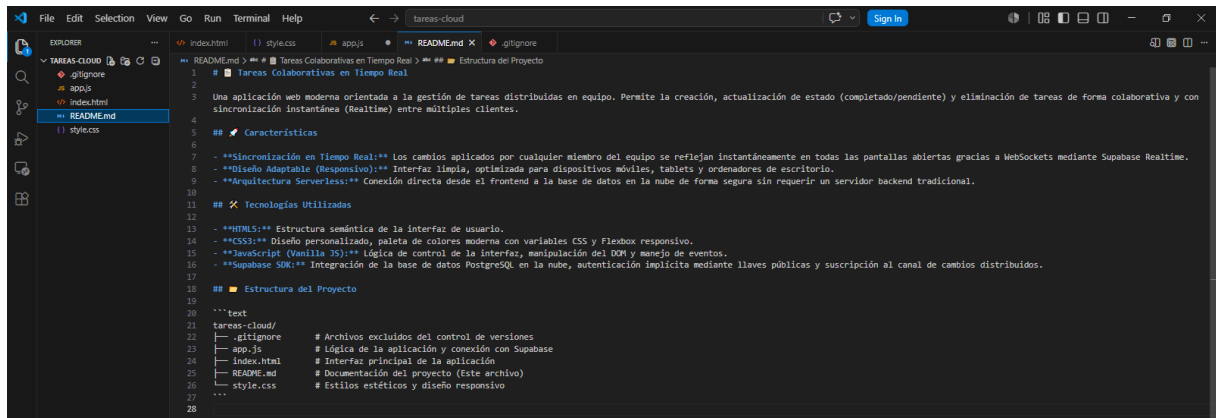
The screenshot shows the VS Code editor interface. The Explorer sidebar on the left lists the project files: `.gitignore`, `app.js`, `index.html`, `README.md`, and `style.css`. The `style.css` file is selected and its content is displayed in the main editor area. The CSS code defines a root selector with background and text styles, a global reset, a body selector with background and color variables, a header selector with a gradient background, and a main selector with max-width and margin.

```

1  :root {
2    --bg-main: #121318;
3    --bg-card: #1a1c24;
4    --bg-input: #232733;
5    --text-primary: #f3f4f6;
6    --text-muted: #9ca3af;
7    --color-accent: #10b981;
8    --color-accent-hover: #059669;
9    --color-danger: #ef4444;
10   --color-danger-hover: #dc2626;
11   --color-border: #2d313f;
12   --font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
13 }
14
15 * {
16   margin: 0;
17   padding: 0;
18   box-sizing: border-box;
19 }
20
21 body {
22   background: var(--bg-main);
23   color: var(--text-primary);
24   font-family: var(--font-family);
25   padding: 20px;
26 }
27
28 header {
29   text-align: center;
30   margin-bottom: 30px;
31   padding: 20px;
32   background: linear-gradient(135deg, #1e1b4b, #064e3b);
33   border-radius: 12px;
34 }
35
36 header h1 {
37   margin-bottom: 8px;
38 }
39
40 .subtitulo {
41   color: #a7f3d0;
42 }
43
44 main {
45   max-width: 800px;
46   margin: 0 auto;
47 }
48

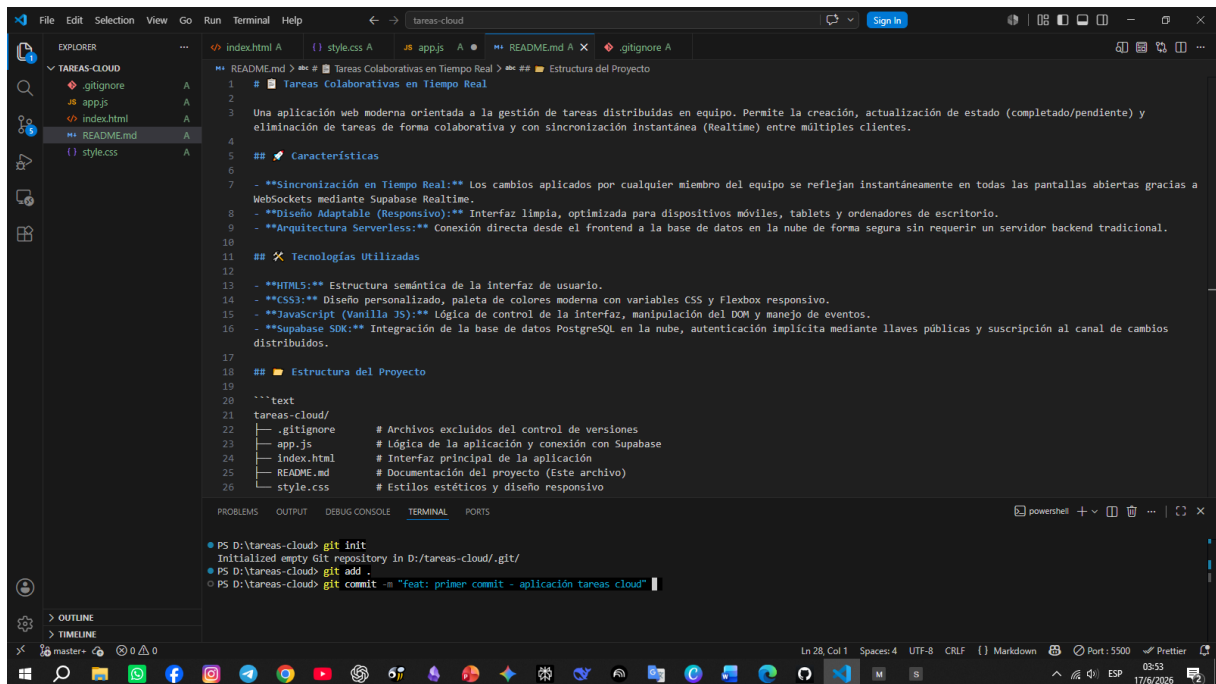
```

Creamos el archivo README.md, donde documentamos el proyecto incluyendo su descripción, las tecnologías utilizadas, las instrucciones de instalación y las capturas de pantalla correspondientes

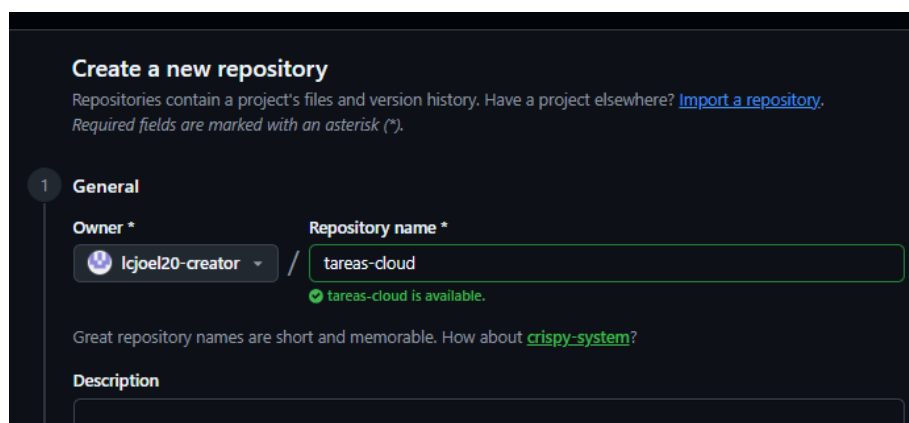


VII. FASE 4: Control de Versiones con Git y GitHub

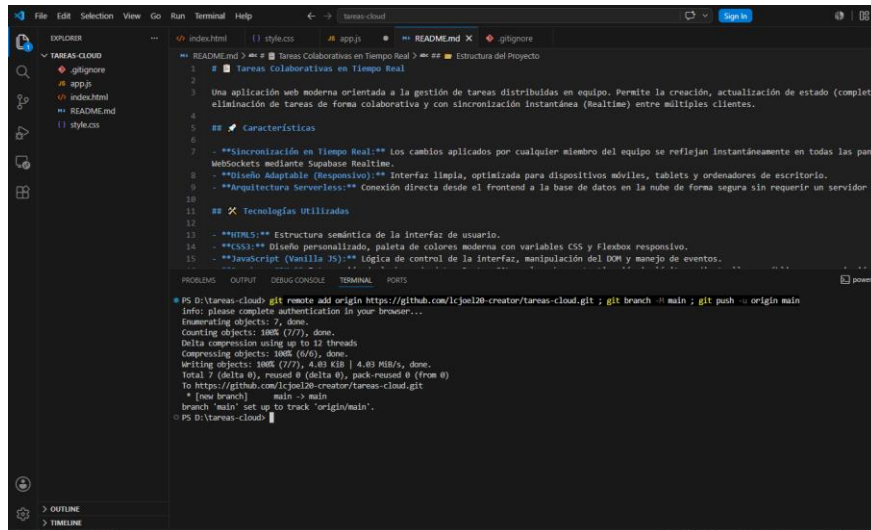
Inicializamos el repositorio Git desde la terminal integrada de VS Code (View > Terminal):



Creamos un nuevo repositorio en GitHub (github.com > New repository). Como nombre sugerido utilizamos tareas-cloud y lo configuramos como público.

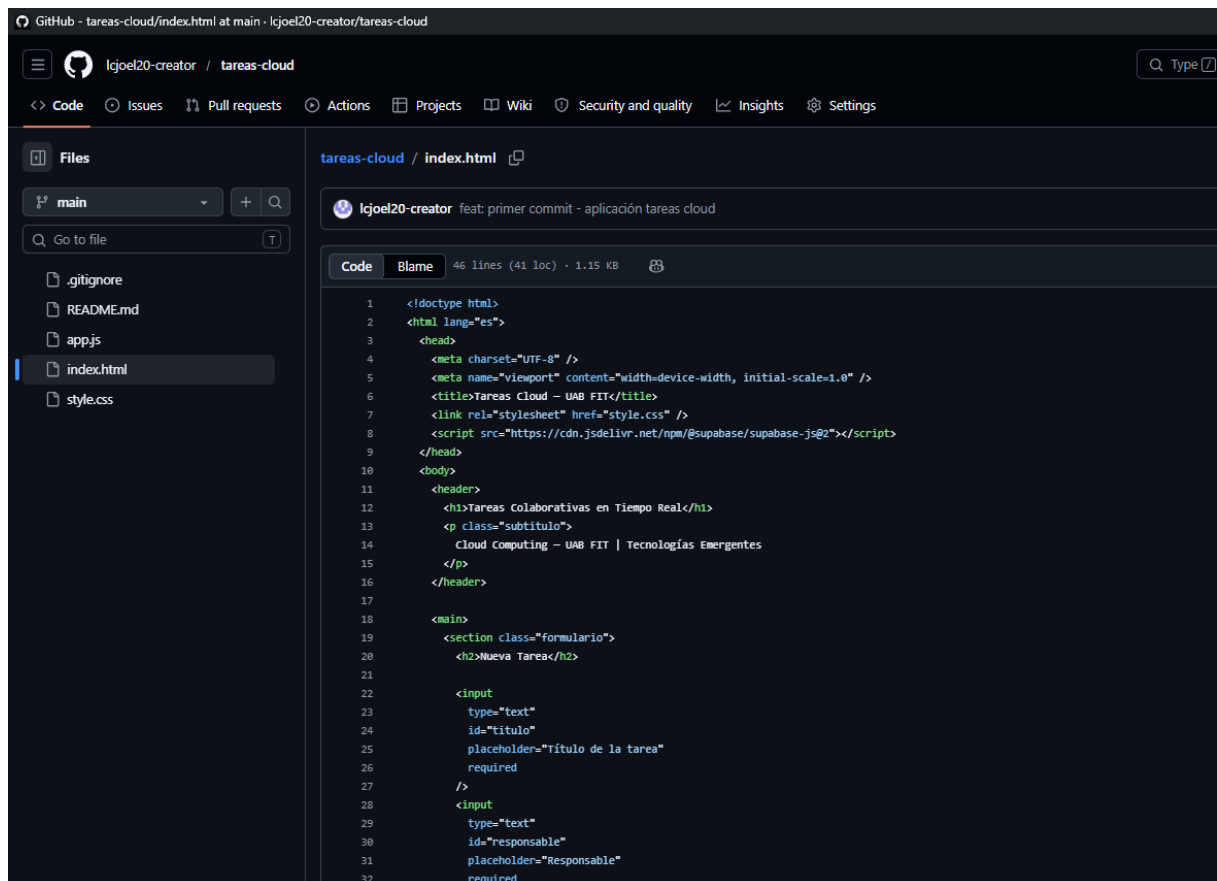


Vinculamos el repositorio local con GitHub y subimos el código al repositorio remoto:



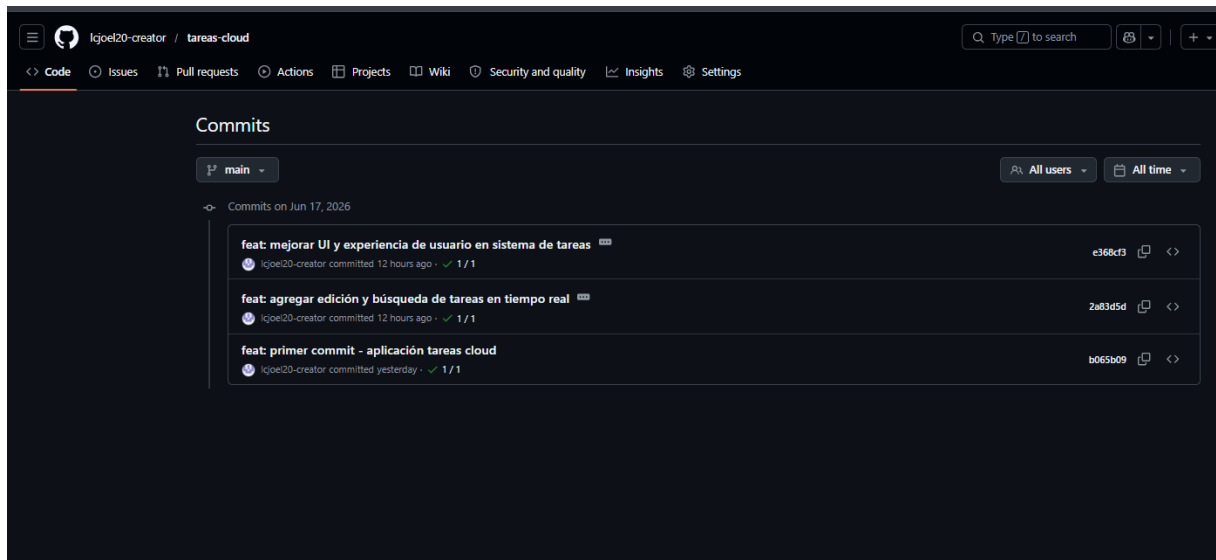
```
PS D:\tareas-cloud> git remote add origin https://github.com/lcjoel20-creator/tareas-cloud.git ; git branch -M main ; git push -u origin main
Info: please complete authentication in your browser...
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (7/7), 4.03 KiB | 4.03 MiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/lcjoel20-creator/tareas-cloud.git
 * [new branch]    main -> main
branch 'main' set up to track 'origin/main'.
PS D:\tareas-cloud>
```

Verificamos que el código se haya subido correctamente y aparezca en el repositorio de GitHub.



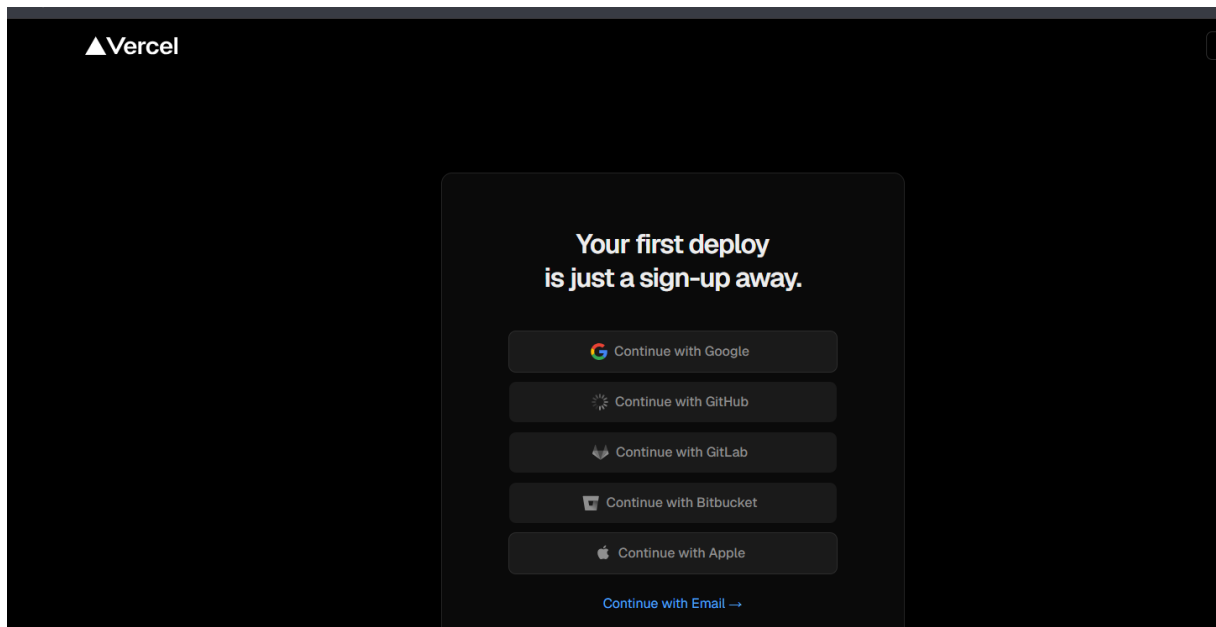
```
1 <!doctype html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6   <title>Tareas Cloud - UAB FIT</title>
7   <link rel="stylesheet" href="style.css" />
8   <script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
9 </head>
10 <body>
11   <header>
12     <h1>Tareas Colaborativas en Tiempo Real</h1>
13     <p class="subtitulo">
14       Cloud Computing - UAB FIT | Tecnologías Emergentes
15     </p>
16   </header>
17
18   <main>
19     <section class="formulario">
20       <h2>Nueva Tarea</h2>
21
22       <input
23         type="text"
24         id="titulo"
25         placeholder="título de la tarea"
26         required
27       />
28
29       <input
30         type="text"
31         id="responsable"
32         placeholder="Responsable"
33         required
```

Realizar al menos 2 commits adicionales con mejoras (estilos, funcionalidades) documentando cada cambio con mensajes descriptivos.

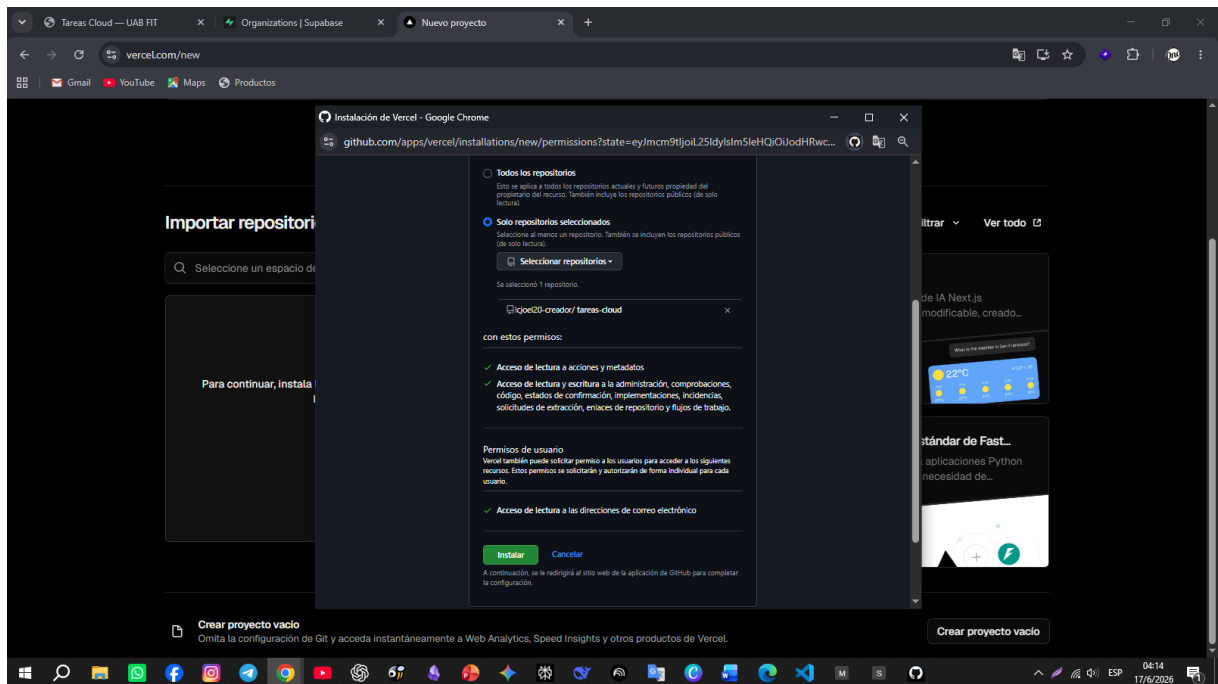


VIII. FASE 5: Despliegue en Vercel (PaaS de Hosting)

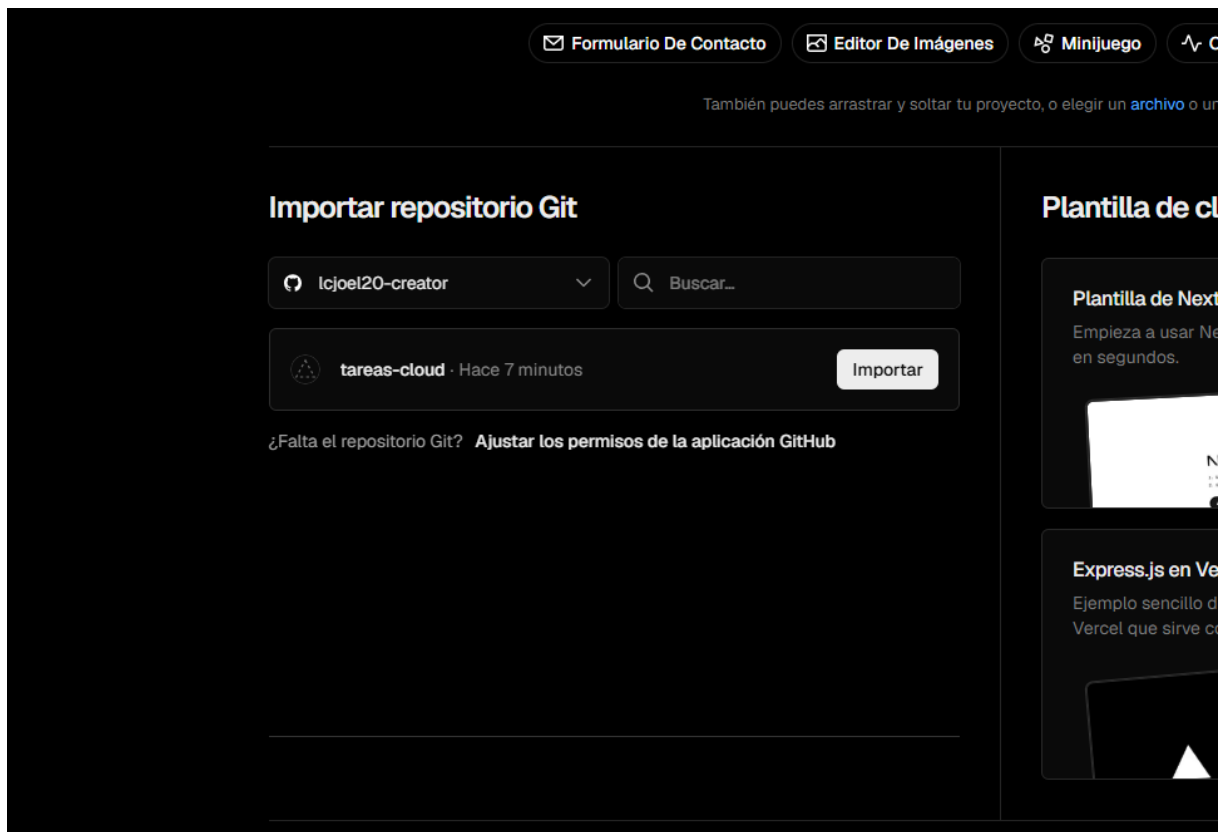
Ingresamos a <https://vercel.com> y creamos una cuenta gratuita, vinculándola con nuestra cuenta de GitHub.



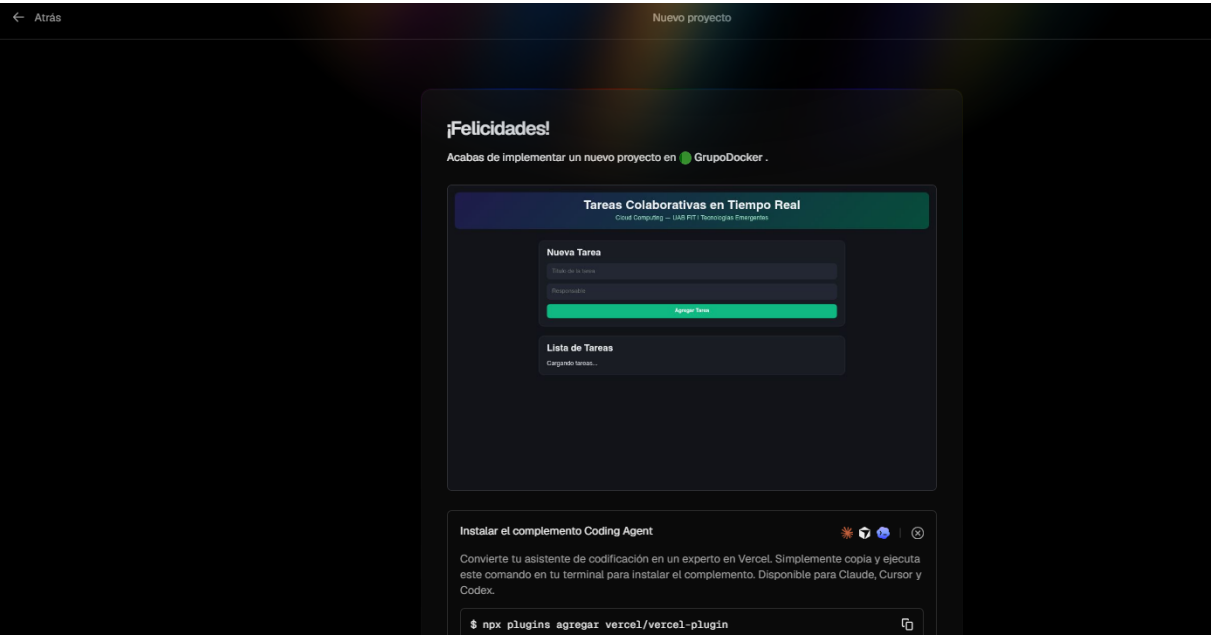
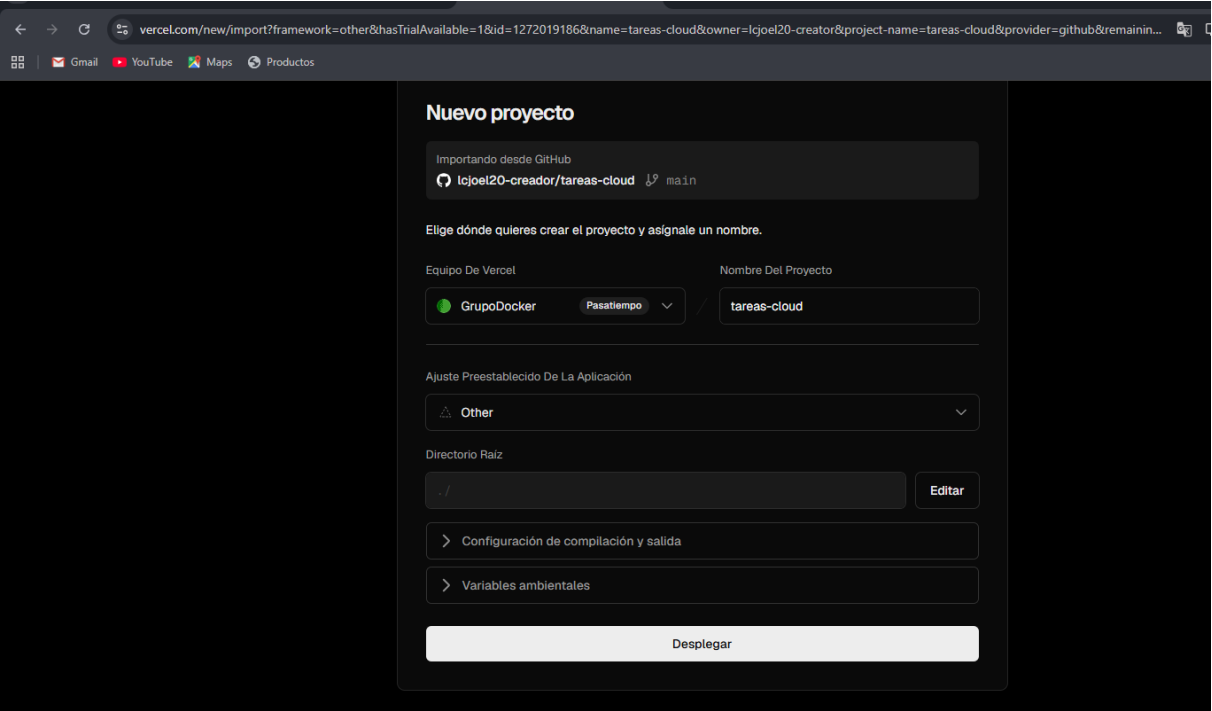
En el dashboard de Vercel, hacemos clic en “Add New Project” y luego en “Import Git Repository”.



Seleccionamos el repositorio tareas-cloud desde GitHub. Vercel detectará automáticamente que se trata de un proyecto HTML estático.

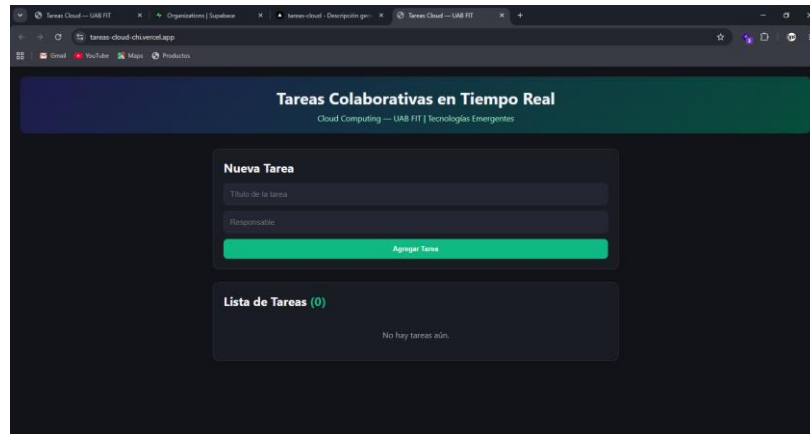


Hacemos clic en “Deploy”. Vercel construirá y desplegará la aplicación automáticamente.



Copiamos la URL de producción generada por Vercel (por ejemplo, tareas-cloud.vercel.app) y probamos la aplicación desde cualquier dispositivo.

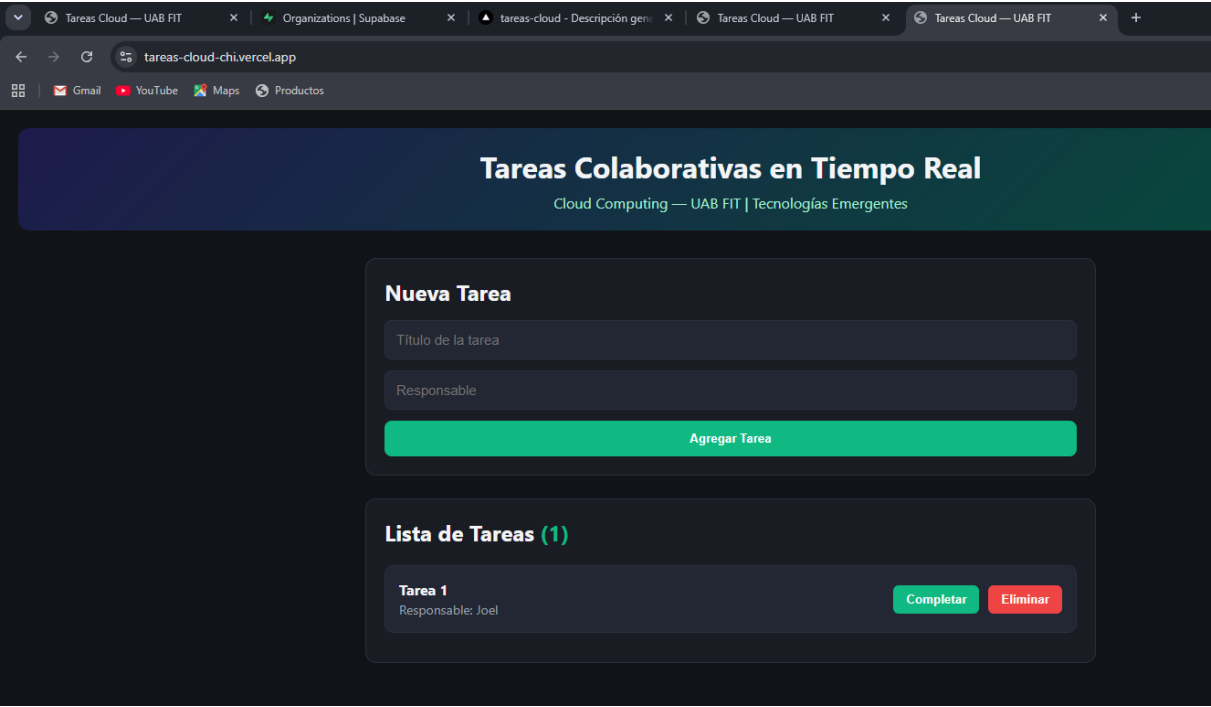
COMPUTADORA



CELULAR



Verificamos que la funcionalidad en tiempo real funciona correctamente abriendo la misma URL en dos pestañas o dispositivos distintos y agregando tareas para comprobar la sincronización.



CELULAR:



IX. REFLEXIÓN

¿En qué modelo de servicio (IaaS/PaaS/SaaS) clasificarías a Supabase y por qué?

Supabase se clasifica como PaaS (Platform as a Service), y más específicamente como BaaS (Backend as a Service). Se considera así porque nos entrega una plataforma completa con herramientas ya listas (base de datos, autenticación, tiempo real) sin que nosotros tengamos que pelearnos configurando el sistema operativo o el hardware del servidor

¿Qué tareas del backend manejó Supabase que normalmente tendrías que hacer manualmente?

Normalmente, tendríamos que instalar y configurar un servidor de base de datos, programar manualmente una API REST (con Node.js o Python) para conectar el frontend con los datos, y configurar un servidor de WebSockets para el tiempo real. Supabase hizo todo eso por nosotros: nos dio la base de datos PostgreSQL y creó automáticamente la API para hacer SELECT, INSERT y DELETE apenas creamos la tabla

¿Cómo funciona el tiempo real en Supabase? ¿Qué tecnología subyacente utiliza?

El tiempo real funciona mediante una tecnología llamada WebSockets. A diferencia de una conexión normal donde el navegador pregunta "¿hay cambios?", aquí el servidor de Supabase "empuja" (push) los cambios directamente al navegador en el instante en que ocurren en la base de datos. En nuestro código, esto lo logramos "suscribiéndonos" a un canal que escucha los eventos de la tabla tareas

¿Cuál fue el proceso de CI/CD que implementaste con GitHub y Vercel?

El flujo de Integración y Despliegue Continuo (CI/CD) que armamos funciona así:

1. Subimos los cambios desde nuestra PC a GitHub usando comandos de Git (git push).
2. Vercel, al estar vinculado con nuestro repositorio de GitHub, detecta ese cambio automáticamente.
3. Vercel toma el código nuevo, lo procesa y lo publica en la URL de producción sin que nosotros tengamos que mover un solo dedo adicional

5. ¿Qué ventajas y desventajas encontraste en el modelo PaaS comparado con un servidor tradicional?

- **Ventajas:** Es mucho más rápido para desarrollar porque la infraestructura ya está gestionada, no pagas por mantenimiento de servidores físicos y tienes escalabilidad automática y seguridad HTTPS de entrada.

- **Desventajas:** Tienes menos control sobre la configuración interna del servidor, dependes totalmente del proveedor (si Supabase cae, tu app también) y las capas gratuitas tienen límites de recursos

¿Qué consideraciones de seguridad deberías aplicar si esta fuera una aplicación en producción real?

Para una aplicación real, no podríamos usar la política de "acceso público" que habilitamos para esta demo. Deberíamos:

- Configurar Row Level Security (RLS) estricto para que solo los dueños de sus tareas puedan verlas o editarlas.
- Activar el módulo de Auth (autenticación) para que los usuarios deban iniciar sesión.
- Manejar las credenciales sensibles como variables de entorno protegidas en Vercel en lugar de dejarlas visibles en el código